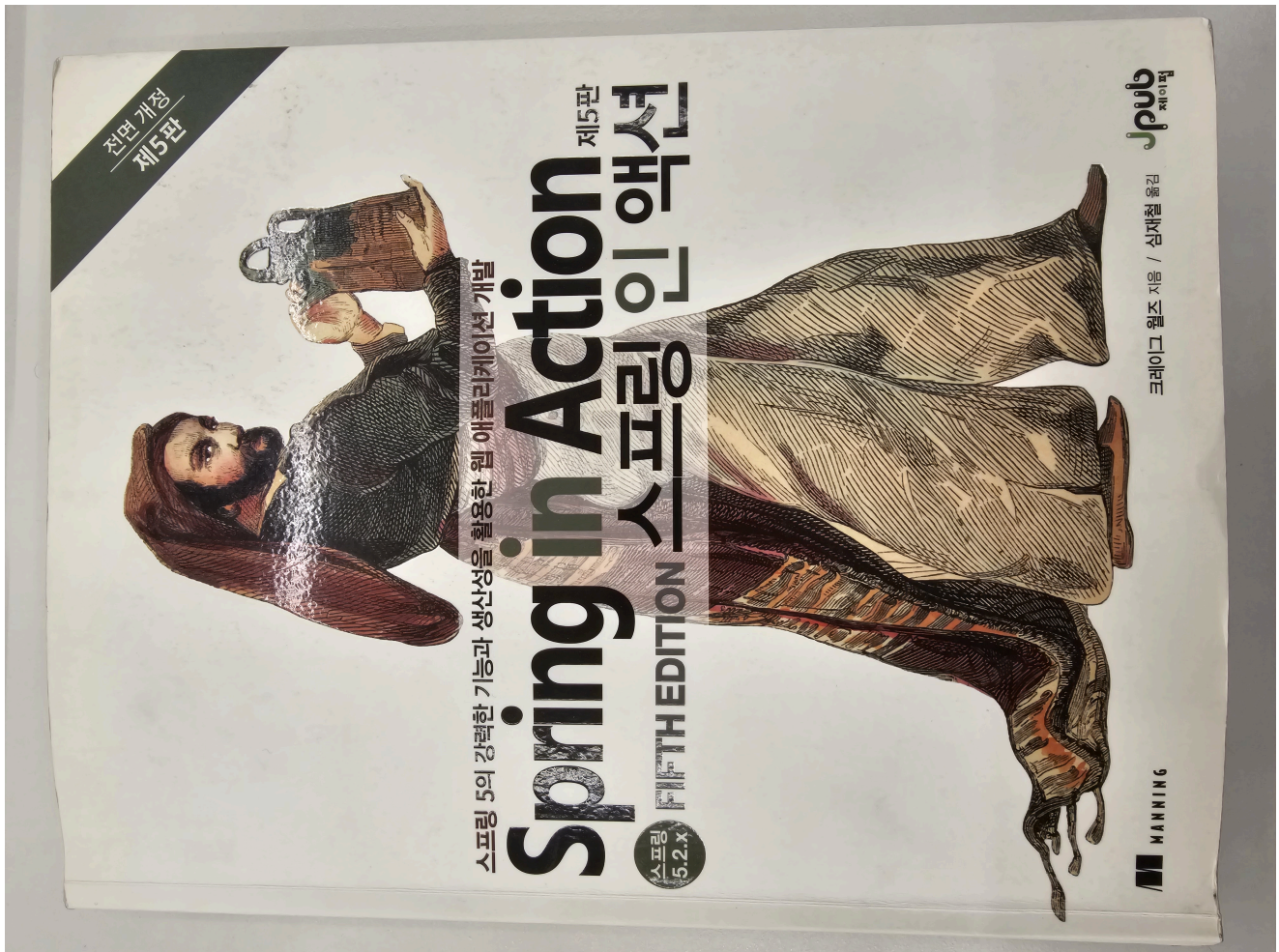


Spring boot - 3

■ Spring in Action 5판 기반 스프링 부트 공부 필기

참고

- 책: *Spring in Action* 제5판 / 크레이그 윌즈 지음, 심재철 옮김
- 코드 및 예시는 책 내용을 기반으로 함
- 나의 학습 환경: Java 17, IntelliJ
- PDF 다운로드: [<https://plusb3b.kr/pdf-download-page/index.html>]
- 책의 공식 GitHub: <https://github.com/Jpub/SpringInAction5>



저장 및 지속성 유지를 타고 클라우드 애플리케이션에 추가할 것이다.

스프링의 JDBC 지원 기능을 사용하는 것 부터 시작하여 더 많은 코드를 줄여서 JPA(Java Persistence API)로 데이터

리퍼지터리를 사용할 수 있도록 할 것이다.

3.1 JDBC를 사용해서 데이터 읽고 쓰기

Java 개발자가 관계형 데이터 사용할 경우 JDBC, JPA 등을 가장 많이 사용하는 두 가지 방법이다. 스프링은 이 두 가지 모두 지원하며, 스프링을 사용하지 않을 때에 비해 더 쉽게 JDBC나 JPA를 사용할 수 있도록 해준다.

스프링의 JDBC 지원은 **JdbcTemplate** 클래스에 기반을 둔다.

JdbcTemplate은 JDBC를 사용할 때 요구되는 모두 형식적이고 상투적인 코드없이 개발자가 관계형 데이터베이스에 대한 SQL 연산을 수행할 수 있는 방법을 제공한다.

예제 1 - JdbcTemplate을 사용하지 않고 데이터베이스 쿼리하기

```
@Repository
public class UserJdbcRawRepository {

    private final DataSource dataSource;

    public UserJdbcRawRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    public User findByName(String name) {
        String sql = "SELECT id, name FROM user WHERE name = ?";
        try {
            Connection conn = dataSource.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)
        ) {
            stmt.setString(1, name);
            try (ResultSet rs = stmt.executeQuery()) {
                if (rs.next()) {
                    User user = new User();
                    user.setId(rs.getLong("id"));
                    user.setName(rs.getString("name"));
                    return user;
                } else {
                    throw new RuntimeException("User not found");
                }
            }
        } catch (SQLException e) {
            throw new RuntimeException("Database error", e);
        }
    }
}
```

예제 2 - JdbcTemplate을 사용하고 데이터베이스 쿼리하기

```
@Repository
public class UserJdbcRepository {

    private final JdbcTemplate jdbcTemplate; //명령문, DB연결코드 x, 객체 클린업 x
```

```

public UserJdbcRepository(JdbcTemplate jdbcTemplate) {
    this.jdbcTemplate = jdbcTemplate;
}

public User findByName(String name) {
    String sql = "SELECT id, name FROM user WHERE name = ?";
    return jdbcTemplate.queryForObject(
        sql,
        new Object[]{name},
        (rs, rowNum) -> {
            User user = new User();
            user.setId(rs.getLong("id"));
            user.setName(rs.getString("name"));
            return user;
        }
    );
}
}

```

예제 1에서 어딘가에 데이터베이스를 쿼리하는 코드가 있다. 그러나 어디있는 지 보려면 JDBC 건초더미에서 쿼리 바늘을 찾느라 헤맬 것이다.

데이터 베이스는 **연결(Connection)**, **명령문 생성(Statement)**, **연결과 명령문 및 결과 세트(ResultSet)** 을 닫고 클린업 하는 코드들로 쿼리 코드가 둘러쌓여기 때문이다.

예제 2에서는 **예제1**보다 훨씬 간단하다.

명령문이나 데이터베이스 연결 객체를 생성하는 코드가 아예 없다. 그리고 메소드의 실행이 끝난 후 그런 객체들을 클린업하는 코드 또한 없다.

쿼리를 수행하고(JdbcTemplate의 queryForObject() 메소드)에 초점을 둘 수 있다.

예제1: Connection, PreparedStatement, ResultSet 객체들을 선언 + 자원해제 코드 존재

예제2: 위 3가지 객체들의 선언과 클린업 코드가 없다.

이제부터 JDBC퍼시스턴스를 우리 애플리케이션에 추가하는 데 필요한 단계를 하나씩 나아가보자.

3.1.1 퍼시스턴스를 고려한 도메인 객체 수정하기

객체를 데이터베이스에 저장하자 할 때는 해당 객체를 고유하게 식별해주는 필드를 하나 추가하는 것이 좋다.

기존에 작성한 Ingredient 클래스는 이미 id 필드를 갖고 있지만 Taco, Order에는 id 필드가 없으므로 추가해야한다.

추가적으로 Taco 객체가 언제 생성되었는지, Order 객체가 언제 주문 되었는지 알면 유용하고 객체가 저장된 날짜와 시간을 갖는 필드를 각 객체에 추가할 필요도 있다.

Taco 클래스에 필요한 id와 createdAt 속성 추가하자.

Taco 클래스 수정

```
(생략)
import java.util.Date;

@Data
public class Taco{
    private Long ind;
    private Date createdAt;
    (생략)
```

Order 클래스 수정

```
(생략)
import java.util.Date;

@Data
public class Order{
    private Long ind;
    private Date placedAt;
    (생략)
```

@Data: getter/setter 및 생성자는 런타임 시에 Lombok이 자동 생성해준다.

간혹 @Data 을 인식하지 못하여 생성이 안되는 경우가 있다. (나처럼 ^^)
그러면 getter/setter, 생성자를 직접 추가를 해야한다

3.1.2 JdbcTemplate 사용하기

JdbcTemplate를 사용하려면 프로젝트의 classpath에 추가해야한다.

이때는 다음과 같은 스프링 부트의 JDBC 스타터 의존성을 빌드명세에 추가하면 간단히 해결된다.

의존성 추가

의존성 - jdbc

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

의존성 - H2내장 DB

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

H2 DB의 경우 버전 정보도 추가해야한다.

버전 정보 추가하기


```
<properties>
  <java.version>17</java.version> <!--원래 있는 거-->
  <h2.version>1.4.196</h2.version><!-- 추가하는 거 -->
</properties>
```

이제 H2 사용해서 데이터 가져오고 저장하는 리퍼지터리 클래스를 작성한다.

리퍼지터리 클래스

- DB와 상호작용 담당
- DB와 통신 담당 + CRUD 작업 수행

JDBC 리퍼지터리 정의하기

내가 만드려고자하는 리퍼지터리의 연산 수행 리스트

- DB의 모든 데이터 쿼리하여 객체 컬렉션(예를 들어List)에 넣기
- Id 사용해서 하나의 객체를 쿼라한다
- 객체를 DB에 저장한다

이 3가지를 메소드로 정의한다. 인터페이스로!

이 3가지 연산을 **IngredientRepository**에 정의한다.

```
public interface IngredientRepository{
    Iterable<Ingredient> findAll();
    Ingredient findById(String id);
    Ingredient save(Ingredient ingredient);
}
```

이 때 Iterable로 선언한 이유는?

여러 개 데이터 반환하는 메소드의 경우 보통 Collection사용한다. : Iterable, List, Set ...*

- 1. 구체적인 컬렉션 타입 지정 안해도 데이터 반복해서 접근 가능
- 2. 데이터가 많으면 DB 한번에 로딩하지 않고 필요할 때마다 가져온다
- 3. 단순 순회목적이면 좋은 선택이다

이제 Ingredient 리퍼지터리가 해야할 일을 IngredientRepository 인터페이스에 정의 했으므로
이제는 JdbcTemplate을 이용해서 데이터베이스 쿼리에 사용할 수 있도록 Repository 인터페이스 구현해야한다.

JdbcIngredientRepository 생성

```
package tacos.data;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
```

```
import org.springframework.stereotype.Repository;

import tacos.Ingredient;
@Repository
public class JdbcIngredientRepository {
    private JdbcTemplate jdbc;

    @Autowired
    public JdbcIngredientRepository (JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }
}
```

1. **@Repository** : **JdbcIngredientRepository** 클래스에 **@Repository** 지정하면 자동으로 찾아서 스프링 애플리케이션 컨텍스트의 빈으로 생성한다.

2. **@Autowired**: 스프링이 해당 빈을 **JdbcTemplate** 에 주입한다.

다시말하자면 **JdbcIngredientRepository** 클래스에 **@Repository**를 지정하면, 스프링 컴포넌트 검색에서 이 클래스를 자동으로 찾아서 스프링 애플리케이션 컨텍스트의 빈으로 생성해준다.

private JdbcTemplate jdbc : DB통신을 위해 사용한다

생성자: DB접근 로직 담당하며 생성자 통해서 **JdbcTemplate** 객체 주입 받는다.

JdbcIngredientRepository 빈이 생성되면 **@Autowired** 애노테이션을 통해서 스프링이 해당 빈을 **JdbcTemplate**에 주입한다.

JdbcIngredientRepository의 생성자에서는 **JdbcTemplate** 참조를 인스턴스 변수에 저장한다.

추가정보

스테레오타입 애노테이션은 스프링에서 주로 사용하는 **역할 그룹**을 나타내는 애노테이션이다.

예를 들어 **@Component**는 스프링이 자동으로 탐색하여 생성하는 빈으로 특정 클래스를 지정하는 클래스 수준의 애노테이션이며.

@Repository는 **@Component** 에서 특화된 **데이터 액세스 관련** 애노테이션이다.

구현을 이제 해보자

```
package tacos.data;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

import tacos.Ingredient;
@Repository
public class JdbcIngredientRepository implements IngredientRepository {
    private JdbcTemplate jdbc;

    @Autowired
    public JdbcIngredientRepository (JdbcTemplate jdbc) {
```

```

        this.jdbc = jdbc;
    }

    @Override //1번 메소드
    public Iterable<Ingredient> findAll() {
        return jdbc.queryForObject(
            "SELECT id, name, type FROM Ingredient",
            this::mapRowToIngredient, id
        );
    }

    @Override //2번 메소드
    public Ingredient findById(String id) {
        return jdbc.queryForObject (
            "SELECT id, name, type FROM Ingredient WHERE id=?",
            this::mapRowToIngredient, id
        );
    }

    //3번 메소드
    private Ingredient mapRowToIngredient(ResultSet rs, int rowNum) throws
    SQLException {
        return new Ingredient (
            rs.getString("id"),
            rs.getString("name"),
            Ingredient.Type.valueOf(rs.getString("type"))
        );
    }
}

```

설명

1번 메소드

- jdbc.query()에서 파라미터가 2개가 있다.
- param 1: SQL 문
- param 2: spring의 RowMapper Interface 를 구현한 메소드
 - : 쿼리로 생성된 결과세트(ResultSet)의 행(Row) 개수만큼 호출
 - : 결과 세트(ResultSet)의 모든 행을 각각의 객체로 생성 후 List에 저장 후 보관한다

2번 메소드

- jdbc.queryForObject는 하나의 객체만 반환한다.
- 맨 마지막 id부분에서는 SQL에 있는 ? 대신 교체되어 쿼리에 사용된다.

데이터 추가하기

JdbcTemplate의 update()메소드는 데이터베이스에 데이터를 추가하거나 변경하는 어떤 쿼리에도 사용될 수 있다.

save() 메소드에서는 데이터베이스에 데이터를 추가한다.

```

@Override
public Ingredient save(Ingredient ingredient) {
    jdbc.update(
        "INSERT INTO Ingredient (id, name , type) VALUES (?, ?, ?)",
        ingredient.getId(),
        ingredient.getName(),
        ingredient.getType().toString()
    );
    return ingredient;
}

```

JdbcTemplate의 update() 메소드는 결과 세트의 데이터를 객체로 생성할 필요가 없다.

query(), queryForObject() 보다 훨씬 간단하다.

update() 메소드에는 수행될 **SQL**을 포함하는 문자열과 쿼리 매개변수에 지정할 값만 인자로 전달한다

즉, **update()** 메소드는 **INSERT, UPDATE, DELETE** 모두 사용할 수 있는 **범용적인 데이터 변경 메소드**이기 때문에

SQL만 바꾸고 매개변수만 맞춰주면 **update**나 **delete** 도 처리할 수 있다.

만약 데이터를 수정하고자하면 메소드명과 내부로직만 살짝 바꿔주면 사용이 가능하다.

이제 JdbcIngredientRepository 가 완성되었으므로, 이제는 이것을 DesignTacoController 에 주입(연결)하고 Ingredient 객체의 List 대신

데이터베이스로부터 읽은 데이터로 생성한 Ingredient 객체의 List를 제공할 수 있다.

```

@Slf4j
@Controller
@RequestMapping("/design")
public class DesignTacoController {

    private final IngredientRepository ingredientRepo;

    @Autowired
    public DesignTacoController (IngredientRepository ingredientRepo) {
        this.ingredientRepo = ingredientRepo;
    }

    @GetMapping
    public String showDesignForm(Model model) {

        List<Ingredient> ingredients = new ArrayList<>(); //객체에 저장할 리스트 생성
        ingredientRepo.findAll().forEach(i -> ingredients.add(i));

        Type[] types Ingredient.Type.values();
        for(Type type : types)
            model.addAttribute(type.toString().toLowerCase(),
                filterByType(ingredients, type));

        model.addAttribute("taco", new Taco());
    }
}

```

```

    return "design";
}

```

ingredientRepo.findAll().forEach(i -> ingredients.add(i));

: 각 **ingredient** 객체를 **ingredients** 리스트에 추가

: **findAll()** 이 반환하는 **Iterable**을 **for-Each** 반복문으로 순회

이제 주입된 IngredientRepository의 findAll() 메소드를 showDesignForm() 메소드에 호출한다
findAll() 메소드는 모든 식자재 데이터를 데이터베이스로부터 가져온다.

그 다음 타입별로 식자재가 필터링 된다

그러나 쿼리에서 참조되는 ingredient 테이블로부터 데이터를 읽기 위해서는 먼저 이 테이블 생성 후 식자재 데이터를 미리 추가해놓아야한다.

3.1.3 스키마 정의하고 데이터 추가하기

- **Ingredient** : 식자재 정보 저장

- **Taco** : 사용자가 식자재를 선택하여 생성한 타코 디자인에 관한 정보 저장한다
- **Taco_Ingredients** : Taco와 Ingredient 테이블 간의 관계를 나타내며, Taco 테이블의 각 행에 대해 하나 이상의 행을 포함한다.
- **Taco_Order** : 주문 정보를 저장한다
- **Taco_Order_Tacos** : Taco_Order와 Taco 테이블 간의 관계를 나타내며, Taco_Order 테이블의 각 행에 대해 하나 이상의 행을 포함한다.

테이블 생성하는 SQL문 정의

Ch01

Ch02

Ch03-JDBC

Ch03-JDBC용-SQL파일

data.sql

schema.sql

Ch03-JPA

Ch04-JDBC인증용-SQL

Ch04

Ch05

Ch06

Ch07

Ch08

Ch09

Ch10

Ch11_12

Ch13

Ch14

Code

Blame

26 lines (25 loc) · 1.07 KB

```

1  delete from Taco_Order_Tacos;
2  delete from Taco_Ingredients;
3  delete from Taco;
4  delete from Taco_Order;
5
6  delete from Ingredient;
7  insert into Ingredient (id, name, type)
8      values ('FLTO', 'Flour Tortilla', 'WRAP');
9  insert into Ingredient (id, name, type)
10     values ('COTO', 'Corn Tortilla', 'WRAP');
11 insert into Ingredient (id, name, type)
12     values ('GRBF', 'Ground Beef', 'PROTEIN');
13 insert into Ingredient (id, name, type)
14     values ('CARN', 'Carnitas', 'PROTEIN');
15 insert into Ingredient (id, name, type)
16     values ('TMT0', 'Diced Tomatoes', 'VEGGIES');
17 insert into Ingredient (id, name, type)
18     values ('LETC', 'Lettuce', 'VEGGIES');
19 insert into Ingredient (id, name, type)
20     values ('CHED', 'Cheddar', 'CHEESE');
21 insert into Ingredient (id, name, type)
22     values ('JACK', 'Monterrey Jack', 'CHEESE');
23 insert into Ingredient (id, name, type)
24     values ('SLSA', 'Salsa', 'SAUCE');
25 insert into Ingredient (id, name, type)
26     values ('SRCR', 'Sour Cream', 'SAUCE');

```

여기 참고해서 작성할 것

이 파일을 어디에 뒤킃할까?

src/main/resources 폴더에 저장하면된다.

이름없는 텍스트 파일을 선택 후 schema.sql, data.sql 을 입력하면 된다.

3.1.4 타코와 주문 데이터 추가하기

지금까지 JdbcTemplate 을 사용해서 데이터베이스에 데이터를 저장하는 방법을 전반적으로 알아보았으므로

JdbcIngredientRepository의 save() 메소드에는

JdbcTemplate의 update() 메소드 사용해서 Ingredient 객체를 데이터베이스 데이터로 저장한다

JdbcTemplate을 사용해서 데이터를 저장하는 방법은 2가지가 있다.

- 직접 **update()** 메소드를 사용한다
- **SimpleJdbcInsert 래퍼 Wrapper** 클래스를 사용한다

JdbcTemplate을 사용해서 데이터 저장하기

우선 타코와 주문 리퍼지터리에서 Taco와 Order객체르 저장하기 위한 인터페이스를 정의해야한다.

Taco 객체를 저장하는데 필요한 TacoRepository인터페이스 정의를 해야한다

TacoRepository

```
package tacos.data;

import tacos.Taco;

public interface TacoRepository {
    Taco save(Taco design);
}
```

OrderRepository

```
package tacos.data;

import tacos.Order;

public interface OrderRepository {
    Order save(Order order);
}
```

사용자가 식자재를 선택하여 생성한 타코 디자인을 저장하려면 해당 타코와 연관된 식자재 데이터도 **Taco_Ingredients 테이블**에 저장해야함.

마찬가지로 주문을 저장하려면 해당 주문과 연관된 타코 데이터를 **Taco_Order_Tacos 테이블**에 저장해야한다.

해당 주문에 어떤 타코들이 연관된 것인지 알 수 있어야 하기 때문이다.

TacoRepository를 구현하려면 다음 일을 수행하는 **save()** 메소드를 구현해야한다.

즉, 타코 디자인 정보를 저장한 다음 **Taco 객체 id**와 이 객체의 **List**에 저장된 각 **Ingredient**객체 **id**를 **Taco_Ingredient** 테이블의 행으로 추가한다.

```
package tacos.data;

import java.sql.*;
import util.*;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.PreparedStatementCreator;
import org.springframework.jdbc.core.PreparedStatementCreatorFactory;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.jdbc.support.KeyHolder;
import org.springframework.stereotype.Repository;

import tacos.Ingredient;
import tacos.Taco;

@Repository
public class JdbcTacoRepository implements TacoRepository {

    private JdbcTemplate jdbc;

    public JdbcTacoRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    //메소드 1번
    @Override
    public Taco save(Taco taco) {
        long tacoId = saveTacoInfo(taco);

        taco.setId(tacoId);
        for(Ingredient ingredient : taco.getIngredients()) {
            saveIngredientToTaco(ingredient, tacoId);
        }

        return taco;
    }

    //메소드 2번
    private long saveTacoInfo(Taco taco) {
        taco.setCreatedAt(new Date());

        // 1. SQL 쿼리 정의
        String sql = "INSERT INTO Taco (name, createdAt) VALUES (?, ?)";

        // 2. PreparedStatementCreatorFactory를 사용해 파라미터 타입 지정
        PreparedStatementCreatorFactory pscFactory =
            new PreparedStatementCreatorFactory(sql, Types.VARCHAR,
            Types.TIMESTAMP);
```

```

// 3. auto-generated key를 받기 위해 설정
pscFactory.setReturnGeneratedKeys(true);

// 4. 파라미터 값 설정
PreparedStatementCreator psc = pscFactory.newPreparedStatementCreator(
    Arrays.asList(
        taco.getName(),
        new Timestamp(taco.getCreatedAt().getTime())
    )
);

// 5. 키 홀더를 생성하고 JdbcTemplate으로 insert 실행
KeyHolder keyHolder = new GeneratedKeyHolder();
jdbc.update(psc, keyHolder);

// 6. 생성된 ID 반환
return keyHolder.getKey().longValue();
}

//메소드 3번
private void saveIngredientToTaco(Ingredient ingredient, long tacoId) {
    jdbc.update(
        "INSERT INTO Taco_Ingredients (taco, ingredient)" +
        "VALUES (?, ?)",
        tacoId, ingredient.getId());
}
}

```

JdbcTacoRepository 설명

@Repository

: Spring이 이 클래스를 DB와 연결된 Repository로 인식 + 예외처리(SQLException) 자동

private JdbcTemplate jdbc;

: Jdbc 간편하게 사용할 수 있도록 하는 Spring 의 클래스

: SQL 실행 + 데이터 조회/업데이트 등 쉽게 처리

생성자

:JdbcTemplate을 Spring이 자동으로 주입

메소드 1번 : save()

: Taco 객체를 DB에 저장하는 메소드

- **1. long tacold = saveTacoInfo(taco);**
: saveTacoInfo() 호출하여 Taco 정보 저장 후, ID를 tacold에 저장한다
- **2. taco.setId(tacold);**
: 객체에 저장된 ID 설정
- **3. for문**
: Taco 클래스의 ingredients는 List 형태 ingredientId 를 기반으로 Ingredient 객체 찾는다

: 찾은 후, saveIngredientToTaco()로 저장한다.

메소드 2번 : saveTacoInfo()

: 생성된 타코ID 제송: KeyHolder.

: KeyHolder 사용하기 위해 PreparedStatementCreator 생성

- **taco.setCreatedAt(new Date());**
: Taco 객체 생성시간을 현재 시간으로 설정
- **PreparedStatementCreator**
: Spring JDBC에서 SQL쿼리를 실행할 때 사용할 PreparedStatement 객체를 생성하는 역할을 하는 인터페이스이다.
: 일반적으로 JdbcTemplate 사용할 때 파라미터가 있는, SQL 쿼리를 실행할 때 유용하게 사용된다.
- **pscFactory.setReturnGeneratedKeys(true);**
: 쿼리 실행 시 생성된 키를 DB에서 받아온다고 JDBC에 알려줌
: 이거 안하면 DB에서 생성된 키를 JDBC가 무시하고 버려버림
- **new PreparedStatementCreatorFactory(sql, Types.VARCHAR, Types.TIMESTAMP);**
: SQL문, ?의 타입 지정1, ?의 타입지정2
- **pscFactory.setReturnGeneratedKeys(true);**
: DB가 자동 생성한 키(예: AUTO_INCREMENT id)를 돌려받기 위해 설정
- **public PreparedStatementCreator = newPreparedStatementCreator(List<?> parameters)**
: ?에 순서대로 바인딩 해준다
: newPreparedStatementCreator(...)는 이 쿼리에 값을 넣기 위한 PreparedStatementCreator 객체를 만든다
: Arrays.asList(..)는 실제로 ? 자리에 들어갈 값들을 순서대로 나열한 것
: 리스트로 감싸는 이유는 쿼리마다 ? 갯수가 다를 수 있다
: List.of(taco.getName(), new Timestamp(taco.getCreatedAt().getTime()))도 사용가능
- **KeyHolder keyHolder = new GeneratedKeyHolder();**
: 받을 준비를 실제 객체로 준비하는 것
: 실행된 쿼리에서 생성된 키는 이 객체에 저장된다

keyHolder.getKey().longValue();

: 이를 통해 생성된 키를 꺼낼 수 있다.

KeyHolder keyHolder = new GeneratedKeyHolder();

jdbc.update(psc, keyHolder);

: 매개변수로 psc가 있는데 이는 우리가 작성한 SQL문 + 값이 바인딩된 SQL을 제공한다

: keyHolder은 GeneratedKeyHolder 실행 결과로 생긴 키 값을 여기에 담음

: psc가 만든 PreparedStatement를 DB에 보내고 DB가 SQL문 실행 후 id 같은 키가 자동으로 생성된다.

: JDBC가 그 키 값을 keyHolder에 저장한다

for(Ingredient ingredient : taco.getIngredients()) 에러 해결

: Taco 클래스의 ingredients 속성은 해당 타코와 연관된 식자재들의 id(String 타입) 만 저장하는 List

: 따라서 Ingredient로 바꿔줘야함

TacoRepository를 DesignTacoController에 주입하고 타코를 저장할 때 사용

따라서 DesignTacoController 수정해야한다

(생략)

```
public class DesignTacoController {  
  
    private final IngredientRepository ingredientRepo;  
    private TacoRepository tacoRepo; // 이 부분 추가  
  
    @Autowired  
    public DesignTacoController ( .... , TacoRepository tacoRepo){  
        (생략)  
        this.tacoRepo = tacoRepo; //이 부분 추가  
    }  
}
```

(생략)

showDesignForm()과 processDesign() 메소드에서 사용할 수 있도록 두 인자 모두 인스턴스 변수에 지정한다

이제 타코 디자인 지정하고 주문과 연결시켜야한다

(생략)

```
@SessionAttributes("order")
```

```
public class DesignTacoController {
```

(생략)

```
    @ModelAttribute(name = "order")  
    public Order order() { return new Order(); }
```

```
    @ModelAttribute(name = "taco")  
    public Taco taco() { return new Taco(); }
```

```
    @PostMapping  
    public String processDesign(@Valid Taco design, Errors, @ModelAttribute Order  
order) {  
        if(errors.hasError()) return "design";  
        Taco saved = tacoRepo.save(design);  
        order.addDesign(saved);  
  
        return "redirect:/orders/current";  
    }  
}
```

그리고 이제 Order 클래스에 addDesign() 메소드 작성하기

(생략)

```
public class Order{  
    (생략)  
    private List<Taco> tacos = new ArrayList<>();  
  
    public void addDesign(Taco design) {  
        this.tacos.add(design);  
    }  
}
```

(생략)

```
}
```

@SessionAttributes("order")

: Order 객체를 세션에 저장해서 여러 요청 간에 유지되게 해줌.

@ModelAttribute(name="~~~")

: 컨트롤러에 요청이 들어올 때 모델 객체를 초기화해서 뷰에 넘김.

: 뷰에서 form th:object="\${taco}" 처럼 사용할 수 있음.

@ModelAttribute Order order

: 세션에서 가져온 Order 객체. 유저의 현재 주문을 의미함

사용자가 주문 폼에 입력하고 제출할 때까지 Order 객체는 세션에 남아있고 데이터베이스에 저장X
이제는 주문 저장하기 위해 OrderController가 OrderRepository 사용할 수 있어야한다
따라서 OrderRepository 구현하는 클래스 작성해야함

SimpleJdbcInsert를 사용해서 데이터 추가하기

복잡한 PreparedStatementCreator 대신 SimpleJdbcInsert를 사용하는 방법 보여준다.

SimpleJdbcInsert는 데이터를 더 쉽게 테이블에 추가하기 위해 JdbcTemplate을 래핑한 객체이다

1) OrderRepository 구현하는 JdbcOrderRepository 생성

: Taco_Order, Taco_Order_Tacos 테이블에 데이터 추가를 위해 두 개의 SimpleJdbcInsert 인스턴스 생성한다.

```
@Repository
public class JdbcOrderRepository implements OrderRepository{
    private SimpleJdbcInsert orderInserter;
    private SimpleJdbcInsert orderTacoInserter;
    private ObjectMapper objectMapper;

    @Autowired
    public JdbcOrderRepository (JdbcTemplate jdbc){
        this.orderInserter = new SimpleJdbcInsert(jdbc)
            .withTableName("Taco_Order")
            .usingGeneratedKeyColumns("id");

        this.orderTacoInserter = new SimpleJdbcInserter(jdbc)
            .withTableName("Taco_Order_Tacos");

        this.objectMapper = new ObjectMapper();
    }
}
```

private ObjectMapper objectMapper;

: JSON 데이터 다룰 때 사용하는 Jackson 라이브러리의 ObjectMapper 객체. 보통 Java 객체를 Map이나 JSON으로 변환 시 사용한다.

@Autowired

: Spring이 자동으로 JdbcTemplate 주입한다.

.withTableName()

: 대상 테이블 지정한다

.usingGeneratedKeyColumns()

: id컬럼이 자동 생성되는 PK임을 설정한다

this.objectMapper = new ObjectMapper();

: JSON데이터 다룰 수 있게 ObjectMapper 생성한다

save() 메소드에서 SimpleJdbcInsert 인스턴스 사용

```
@Override
public Order save (Order order){
    order.setPlacedAt(new Date()); //현재 시간 설정
    long orderId = saveOrderDetails(order); //Taco_Order테이블에 주문 삽입 후, 생성된
주문 아이디 받음
    order.setId(orderId); // 생성된 orderId를 order객체에 설정
    List<Taco>tacos = order.getTacos();

    for(Taco taco : tacos)
        saveTacoOrder(taco, orderId);

    //order객체에서 tacos 리스트 가져온다. for-each통해 각 타코를 주문과 연결해 저장한다

    return order;
}
```

```
private long saveOrderDetails(Order order){

    @SuppressWarnings("unchecked")
    Map<String, Object> values = objectMapper.convertValue(order, Map.class);
    values.put("placedAt", order.getPlacedAt());

    long orderId = orderInserter
        .executeAndReturnKey(values)
        .longValue();

    return orderId;
}
```

@SuppressWarnings("unchecked")

: objectMapper.convertValue()가 unchecked 경로 발생하므로 무시하도록 설정

convertValue()

: ObjectMapper 사용하여 Order 객체를 Map형태로 변환

: 필드 이름을 자동으로 키 변환하여 삽입 할 데이터로 활용하게함

executeAndReturnKey()

: 테이블에 저장 후 생성된 ID가 Number객체로 반환

longValue()

: long타입으로 반환

```
private void saveTacoOrder(Taco taco, long orderId){
    Map<String, Object> values = new HashMap<>();
    values.put("tacoOrder", orderId);
    values.put("taco", taco.getId());

    orderTacoInsertter.execute(values);
}
```

orderTacoInsertter.execute(values);

: 이를 호출하여 Taco_Order_Tacos 테이블에 데이터 삽입

save()메소드는 실제로 저장하는 일은 하지 않으며 Order 및 이것과 연관된 Taco 객체들을 저장하는 처리를 총괄한다.

실제로 저장하는 일은 **saveOrderDetails()**와 **saveTacoOrder()** 메소드에 위임한다.

SimpleJdbcInsert 는 데이터를 추가하는 두 개의 유용한 메소드인 **execute()**, **executeAndReturnKey()**를 가지고 있다.

모두 **Map <String, Object>** 를 인자로 받는다.

Map의 키는 데이터가 추가되는 테이블의 열 이름과 대응되며 Map의 값은 해당열에 추가되는 값이다.

Order객체의 속성 값들을 Map항목으로 복사하면 되므로 이런 Map의 생성은 쉽다.

속성 모두가 테이블의 열과 같은 이름을 갖는다. 따라서 **orderDetails()**메소드에서 **Jacksons ObjectMapper**와 이것의 **convertValue()**메소드를 사용해서 Order 를 Map으로 변환한 것이다.

주문 데이터의 모든 속성 값을 갖는 **Map**이 준비되었으므로 **orderInsertter**의 **executeAndReturnKey()** 메소드를 호출할 수 있다.

이 메소드를 호출하면 Taco_Order테이블에 저장된 후 DB에서 생성된 아이디가 Number 객체로 반환된다.

객체를 Map으로 변환하기 위해 ObjectMapper를 사용하는 대신, 간단하게 **orderTacoInsertter**의 **execute()**메소드를 호출하여 데이터를 저장할 수 있다.

OrderController에서 OrderRepository 사용하기

```
...
@SessionAttributes("order")
public class OrderController {

    private OrderRepository orderRepo;
```

```

public OrderController(OrderRepository orderRepo) {

    this.orderRepo = orderRepo;

}

@GetMapping("/current")
public String orderForm() {
    return "orderForm"
}

@PostMapping
public String processOrder(@Valid Order order, Errors errors, SessionStatus
sessionStatus){
    if(errors.hasError) return "orderForm";
}

orderRepo.save(order);
sessionStatus.setComplete();

return "redirect:/";
}
}

```

processOrder()

- :orderRepository의 save() 메소드를 통해서 제출된 Order 객체를 저장한다.
- : 따라서 객체도 세션에 보존되어야한다.
- : 저장된 후 더 이상 세션에 보존할 필요 없음
- : 만약 제거하지 않으면 이전 주문 및 연관된 타코가 세션에 남아있게되어 다음 주문은
이전 주문에 포함되었던 타코 객체들을 가지고 시작하게 될 것
- : SessionStatus를 인자로 전달받아 이것의 setComplete() 메소드를 호출하여 세션을 재설정한다.

oirderForm.html 수정

```

...
<ul>
    <li th:each="taco : ${order.tacos}">
        <span th:text="${taco.name}">taco name</span></li>
</ul>
...

```

데이터의 타입을 변환해주는 Converter class 작성

Spring 의 인터페이스에 정의된 convert() 메소드를 구현한다

- String 타입의 식자재 ID를 사용해서 데이터베이스에 저장된 특정 식자재 데이터를 읽은 후 Ingredient 객체로 변환하기 위해 컨버터가 사용된다

```

@Component
public class IngredientByIdConverter implements Converter<String, Ingredient> {
    private IngredientRepository ingredientRepo;

    @Autowired
    public IngredientByIdConverter(IngredientRepository ingredientRepo){
        this.ingredientRepo = ingredientRepo;
    }

    @Override
    public Ingredient convert(String id) {
        return ingredientRepo.findById(id);
    }
}

```

@Component

: 스프링에 의해 자동생성 및 주입되는 빈으로 생성

implements Converter<String, Ingredient>

: 변환할 값 String

: 반환할 값 Ingredient

3.2 스프링 데이터 JPA를 사용해서 데이터 저장하고 사용하기

가장 많이 알려진 스프링 데이터 프로젝트들은 다음과 같다

- Spring Data JPA: 관계형 데이터베이스의 JPA 퍼시스턴스
- Spring Data MongoDB: 몽고 문서형 데이터베이스의 퍼시스턴스
- Spring Data Neo4j: Neo4j 그래프 데이터베이스의 퍼시스턴스
- Spring Data Redis: Redis key-value 스토어의 퍼시스턴스
- Spring Data Cassandra: 카산드라 데이터베이스의 퍼시스턴스

작업: JDBC 기반 Repository를 Spring Data JPA로 교체한다.

3.2.1 스프링 데이터 JPA를 프로젝트에 추가

의존성 추가

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

```

3.2.2 도메인 객체에 어노테이션 추가하기

```

...
@NoArgsConstructor(access = AccessLevel.PROTECTED, force=true)
@Entity

```

```
public class Ingredient {

    @Id
    private final String id;
    ...
}
```

JPA 개체(Entity) 로 선언하려면 반드시 `@Entity` 어노테이션을 추가해야한다.

id 속성에는 반드시 `@Id`를 지정하여 이 속성이 데이터베이스의 개체를 고유하게 식별한다는 것을 나타내야 한다.

JPA에서는 개체가 인자 없는 생성자를 가져야하므로 `@NoArgsConstructor`를 사용한다.

인자없는 생성자의 사용을 원치 않으므로 access속성을 `AccessLevel.PRIVATE` 설정하여 클래스 외부에서 사용하지 못하게 한다

Ingredient에는 초기화가 필요한 final 속성들이 있으므로 force속성을 true로 설정한다.

`@Data`

`@Entity`

public class Taco{

```
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    ...

    @ManyToMany(targetEntity = Ingredient.class)
    private List<Ingredient> ingredients;

    @PrePersist
    void placedAt() {
        this.placedAt = new Date();
    }

}
```

@GeneratedValue(strategy = GenerationType.AUTO)

: JPA에서 기본 키 값을 자동으로 생성.

: PK값 입력하지 않아도 자동 생성

- AUTO: 기본 값. DB에 맞게 자동 선택
- IDENTITY: Auto Increment 사용
- SEQUENCE : 시퀀스 사용
- TABLE: 키 생성 전용 테이블

@ManyToMany(targetEntity = Ingredient.class)

: jpa에서 N:M 관계 매핑할 때 사용.

: 1개의 엔티티가 여러 다른 엔티티와 관계 맺고 그 반대도 성립하는 경우 사용

@PrePersist

: JPA 엔티티가 DB에 삽입되기전 실행되는 메소드에 붙는 어노테이션

추가 참고사항

외래키를 보유한 쪽에서

→ @OneToOne + @JoinColumn(name = "...") 사용

반대쪽 (User)에서는

→ @OneToOne(mappedBy = "...") 사용하여 양방향 연관관계 구성

4. 엔티티 연관관계

본 프로젝트에서는 RDB 기반의 테이블 구조를 Java의 JPA 엔티티로 매핑하면서, 실질적인 객체 간 연관성을 정확히 반영하도록 하였다.

특히 외래키(FK)를 기준으로 연관관계를 분석하고, @ManyToOne, @OneToMany(mappedBy = "..."), @OneToOne 등의 어노테이션을 적절히 적용하여 양방향 혹은 단방향 관계를 명확히 표현하였다.

4.1 대표적인 매핑 예시

Posts(게시글)는 하나의 Board(게시판)에 소속된다

- Posts 엔티티에서 @ManyToOne으로 Board 참조
- Board 엔티티에서는 @OneToMany(mappedBy = "board")로 List<Posts> 선언
- 이를 통해 양방향 관계 형성

하나의 게시판은 여러 게시글을 가질 수 있지만,
하나의 게시글은 오직 하나의 게시판에만 소속될 수 있다.
따라서 Posts 엔티티는 @ManyToOne으로 Board를 참조하고,
Board 엔티티는 @OneToMany(mappedBy = "board")로 List<Posts>를 선언하여
양방향 연관관계를 형성한다.



4.2 전체 매핑관계 매핑 요약 - 표

관계	방향성	설명
User ↔ Posts	1:N	한 사용자는 여러 게시글을 작성할 수 있음
User ↔ Comments	1:N	한 사용자는 여러 댓글을 작성할 수 있음
Posts ↔ Comments	1:N	하나의 게시글에는 여러 댓글이 달릴 수 있음
Posts ↔ PostFiles	1:N	하나의 게시글에 여러 첨부파일이 포함될 수 있음
Board ↔ Games	1:N	게시판은 여러 게임을 가질 수 있음 (게임 게시물 전용 게시판)
User ↔ Games	1:N	한 사용자가 여러 게임을 업로드할 수 있음
User ↔ UserScore	1:N	사용자는 여러 점수 유형을 가질 수 있음 (예: total, weekly)
User ↔ UserLoginInfo	1:N	사용자의 로그인 이력이 누적 기록됨
User ↔ UserAuth	1:1	로그인용 인증 정보 (아이디, 비밀번호)와 1:1 매핑
User ↔ UserPrivate	1:1	민감 정보(이메일, 생년월일 등)와 1:1 매핑

4.3 1:1 관계 설계 기준

- 외래키를 보유한 쪽(예: UserAuth, UserPrivate)
@OneToOne + @JoinColumn(name = "...") 사용
- 반대쪽 (User)
@OneToOne(mappedBy = "...") 사용하여 양방향 연관관계 구성

이 구조는 JPA에서의 식별자 공유 전략 (@MapsId)도 활용하여,
UserPrivate처럼 FK = PK인 경우도 정확히 구현하였다.

```
@Table(name = "user_private")
public class UserPrivate {
    // PK이자 FK인 필드
    @Id
    @Column(name = "pri_ab_id")
    private Long priAbId;

    // Board 1:1 매핑, 외래키 + PK 공유
    @OneToOne
    @MapsId // priAbId = user.ab_id (식별자 공유)
    @JoinColumn(name = "pri_ab_id")
    private User user;
```

4.2 데이터 일관성 유지 전략

모든 관계에서 데이터의 생명주기를 통일하기 위해 다음 설정을 적용하였다

- cascade = CascadeType.ALL
: 연관 객체도 함께 저장/삭제되도록 처리
- orphanRemoval = true
: 관계가 끊긴 자식 엔티티는 자동 삭제되어 데이터 유실 방지

```
11 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
12 private List<Post> ab_posts = new ArrayList<>();
13
14 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
15 private List<Comment> comments = new ArrayList<>();
16
17 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
18 private List<UserScore> user_scores = new ArrayList<>();
19
20 @OneToOne
21 private List<UserLoginInfo> user_login_info = new ArrayList<>();
22
23 @OneToOne(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
24 private UserAuth user_auth;
25
26 @OneToOne(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
27 private UserPrivate user_private;
```

```

...
@Table(name="Taco-Order")
public class Order implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    ...
}

```

private static final long serialVersionUID = 1L;

- : Java 직렬화에서 객체의 버전 식별하는 고유 아이디
- : 직렬화된 객체를 역직렬화할 때 버전 동일한 지 검사
- : 클래스가 변경되어도 serialVersionUID 같으면 역직렬화 가능
- : 다르면 예외발생

직렬화 : 객체를 바이트스트림으로 변환하여 파일 혹은 네트워크로 저장 및 전송하는 과정

역직렬화: 바이트스트림을 다시 객체로 변환

3.2.3 JPA Repository 선언하기

Spring Data에서는 **CrudRepository** 인터페이스 확장이 가능하다

```

public interface IngredientRepository extends <Ingredient, String> {
}

```

Ingredient: Repository에 저장되는 개체 타입

String: 개체 아이디 속성의 타입

CrudRepository 인터페이스를 확장한 3개의 인터페이스를 선언했으므로, **CrudRepository** 인터페이스에 정의된 많은 메소드의 구현을 포함해서 3개의 인터페이스를 구현하는 클래스를 작성해야한다고 생각할 수 있다.

하지만 그럴 필요 없다. 이것이 바로 JPA 장점

스프링 데이터 JPA가 각 인터페이스 구현체를 자동으로 생성하므로 컨트롤러에 주입만 하면 된다.

부트스트랩 클래스 변경

: src/main/java, tacos 패키지에 있는 TacoCloudApplicaion.java 수정

```

...
@SpringBootApplication
public class TacoCloudApplication {
    public static void main(String [] args) {
        ...
    }

    @Bean

```

```

public CommandLineRunner dataLoader(IngredientRepository repo){
    return new CommandLineRunner() {

        @Override
        public void run(String ... args) throw Exception {
            repo.save(new Ingrdient("FLT0", "Flour Tortilla", Type.WRAP);
            ...
        }
    };
}

```

: 애플리케이션 시작되면서 호출되는 dataLoader() 메소드에서 데이터를 미리 DB에 저장할 필요 있음

CommandLineRunner 사용이유

: 테스트 및 초기 데이터 로드

: 애플리케이션 실행 시 자동으로 특정 로직 실행 가능

: 휘발성 DB 사용 시 유용

컨버터 변경

```

...
@Override
public Ingredient convert(String id){
    Optional<Ingredient> optionalIngredient = ingredientRepo.findById(id);
    return optionalIngredient.isPresent()?
        optionalIngredient.get() : null;
}
...

```

Optional

: null 값을 직접 반환하는 것을 방지하는 래퍼 클래스

: isPresent() 통해 값이 존재하는 지 확인한다.

: true면 get() 사용해 객체 반환

: 값 없으면 null 반환

[Spring boot - 2](#)

- 7월.

@Override

```
public Iterable <Ingredient> findAll() {
```

param① : SQL문

```
    return jdbc.query ("select id, name, type from Ingredient",  
        this :: mapRowToIngredient);
```

```
}
```

param② : spring의 RowMapper IF를 구현한 메소드.

: 쿼리 생성된 결과셋(ResultSet)의 행(row) 개수만큼 돌.

: 결과셋의 모든 행을 각각 객체로 생성 후 List에 저장 후 반환.

@Override

```
public Ingredient findById(String id) {
```

```
    return jdbc.queryForObject ( → 하나만 객체만 반환
```

```
        "select id, name, type From Ingredient where id = ?",
```

```
        this :: mapRowToIngredient, (id));
```

↳ SQL에 있는 ? 대신 교체될 쿼리에 사용.

```
}
```

```
private Ingredient mapRowToIngredient (ResultSet rs, int rowNum)
```

```
throws SQLException {
```

```
    return new Ingredient (
```

```
        rs.getString ("id"),
```

```
        rs.getString ("name"),
```

```
        Ingredient.Type.valueOf (rs.getString ("type")));
```

```
}
```

@Override

```
public Ingredient save (Ingredient ingredient) {
```

```
    jdbc.update ( → 데이터를 객체로 생성할 필요X.
```

```
        "insert into Ingredient (id, name, type) values (?, ?, ?)",
```

```
        ingredient.getId(),
```

```
        ingredient.getName(),
```